Backend pour le suivi des élèves - Step7

Maintenant que nous avons validé les données, nous pouvons nous intéresser à la gestion des erreurs.

Grâce à AdonisJS les erreurs sont déjà gérées dans le fichier `app/exceptions/handler.ts`.

```
import app from '@adonisjs/core/services/app'
import { type HttpContext, ExceptionHandler } from '@adonisjs/core/http'

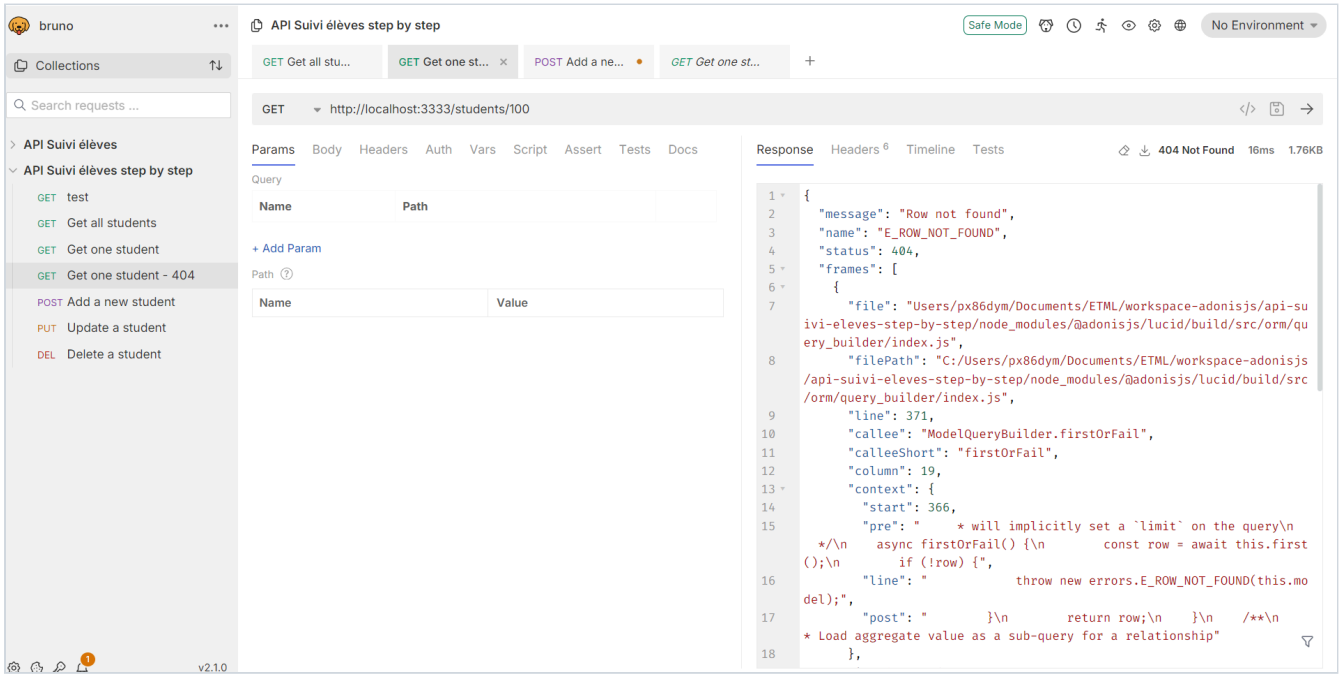
export default class HttpExceptionHandler extends ExceptionHandler {
  /**
   * En mode debug, le détail de l'erreur est retourné.
   * Le mode debug est désactivé en production.
   */
  protected debug = !app.inProduction

  // ...
}
```

> 📝 **À noter**
> Notre API retourne toujours du JSON, même en cas d'erreur.
> C'est le middleware `force_json_response_middleware` (enregistré dans `start/kernel.ts` par le kit starter) qui s'en charge.

Erreur 404

Par exemple, en cas d'erreur 404 :

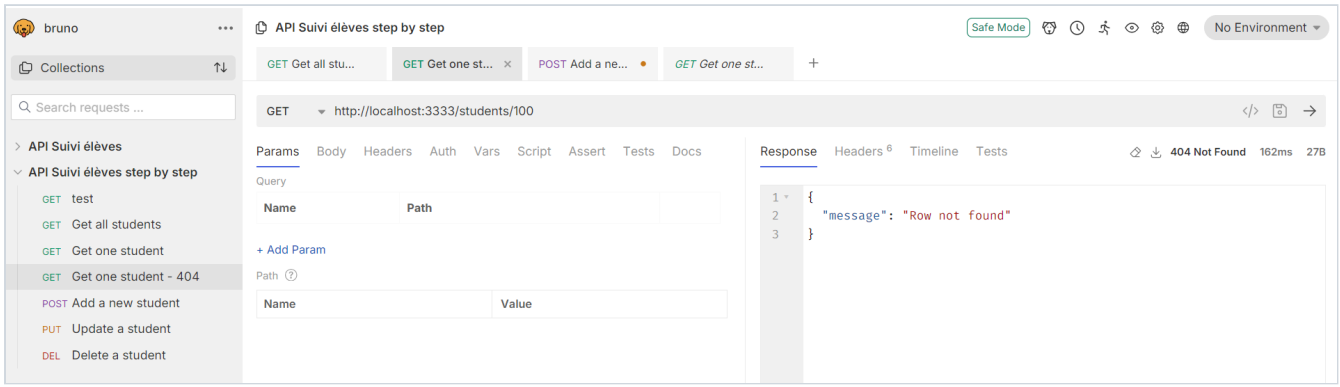


En mode développement, la réponse contient beaucoup d'informations (fichier, ligne, ...) utiles pour corriger notre code.

Modifier le `.env` pour passer en mode PRODUCTION :

```
NODE_ENV=production # passer de development à production
```

Puis redémarrer le serveur et voir la différence d'affichage des erreurs pour un 404.



En mode production, seul le message d'erreur est retourné.

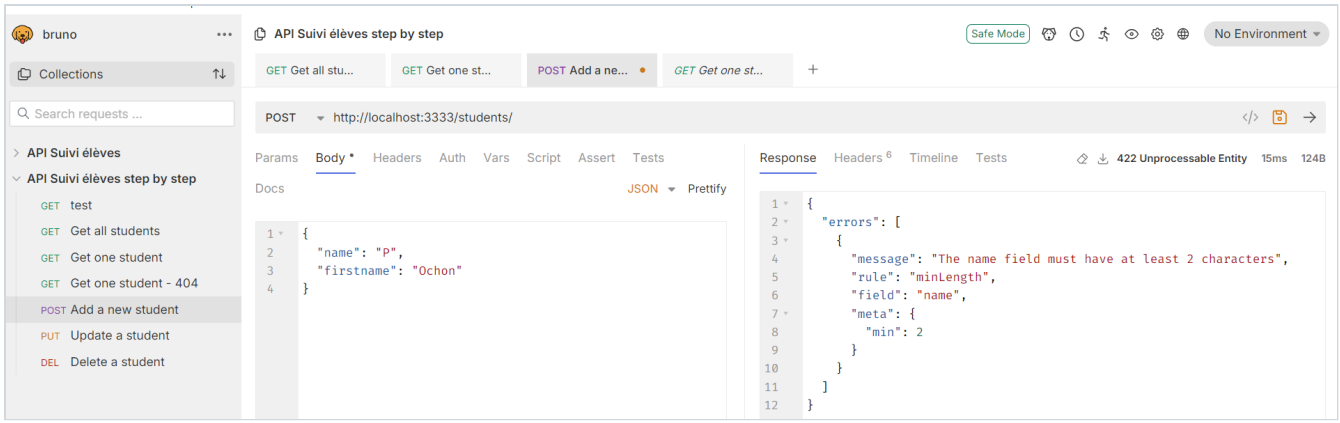
> 💡 **Question**
> Pourquoi ne pas afficher le détail de l'erreur en production ?

Le détail de l'erreur révèle la structure interne de notre application (chemins des fichiers, code source, ...). Ces informations pourraient être utilisées par une personne malveillante.

> ⚠️ **ATTENTION**
> N'oubliez pas de remettre `NODE_ENV=development` dans le `.env` une fois le test terminé.
> En mode production, AdonisJS ne régénère plus le fichier `database/schema.ts` après les migrations et certaines commandes (comme `node ace migration:fresh`) demandent une confirmation supplémentaire.

Erreur de validation

Testons maintenant une erreur lors de la validation :



Le validateur retourne automatiquement un statut HTTP `422` avec la liste des erreurs dans un tableau `errors`.